

Phase 3 — L'agent PyTutor

Quatre concepts essentiels + cinq exercices pour aller plus loin

Cours PyTutor2

Résumé

Cette phase 3 construit un *agent* qui orchestre les briques des phases précédentes via une boucle *Réfléchir-Agir-Observer* (ReAct). On y aborde : le décorateur `@tool` et la docstring comme contrat avec le LLM, la construction de l'agent avec `create_agent` (LangChain 1.x), l'observation de la boucle ReAct sur trois démos contrastées, et la gestion de la mémoire conversationnelle pour un chat multi-tour. Chaque exercice suit la structure habituelle (énoncé + objectifs en boîte, solution détaillée, vérification LangSmith, récapitulatif). Une seconde partie « *Pour aller plus loin* » prolonge avec cinq exercices avancés sur les agents : ajouter un tool dynamique, observer un agent dans LangSmith, paramétrer le `timeout` d'un tool, composer un tool avec un sous-LLM, et streamer la réponse en deux modes.

Table des matières

I Exercices	1
1 Définir des <i>tools</i> avec <code>@tool</code>	2
1.1 Solution	2
1.1.1 Architecture	2
1.1.2 Le code complet	2
1.1.3 Explications pas à pas	5
1.2 Vérification dans LangSmith	7
2 Construire l'agent avec <code>create_agent</code>	8
2.1 Solution	8
2.1.1 Architecture	8
2.1.2 Le code complet	8
2.1.3 Explications pas à pas	9
2.2 Vérification dans LangSmith	11
3 Observer la boucle Réfléchir-Agir-Observer (ReAct)	12
3.1 Solution	12
3.1.1 Architecture	12
3.1.2 Le code complet	12
3.1.3 Explications pas à pas	14
3.2 Vérification dans LangSmith	18
4 Mémoire conversationnelle multi-tours	19
4.1 Solution	20
4.1.1 Architecture	20
4.1.2 Le code complet	20
4.1.3 Explications pas à pas	21
4.2 Vérification dans LangSmith	22

II	Pour aller plus loin	23
5	Ajouter un cinquième tool à l'agent	23
5.1	Solution	23
5.1.1	Architecture	23
5.1.2	Le code complet	24
5.1.3	Explications pas à pas	24
5.2	Vérification dans LangSmith	26
6	LangSmith pour observer un agent	27
6.1	Solution	27
6.1.1	Architecture	27
6.1.2	Le code complet	27
6.1.3	Explications pas à pas	28
6.2	Vérification dans LangSmith	29
7	Tool avec paramètre <code>timeout</code> configurable	29
7.1	Solution	30
7.1.1	Architecture	30
7.1.2	Le code complet	30
7.1.3	Explications pas à pas	31
7.2	Vérification dans LangSmith	33
8	Tool qui appelle un sous-LLM (<code>evaluate_student_answer</code>)	34
8.1	Solution	35
8.1.1	Architecture	35
8.1.2	Le code complet	35
8.1.3	Explications pas à pas	36
8.2	Vérification dans LangSmith	39
9	Streaming avec <code>agent.stream()</code>	40
9.1	Solution	40
9.1.1	Architecture	40
9.1.2	Le code complet	40
9.1.3	Explications pas à pas	41
9.2	Vérification dans LangSmith	43

Première partie

Exercices

1 Définir des *tools* avec `@tool`

Exercice 1. Quatre outils pour l'agent PyTutor

Implémente quatre *tools* que l'agent PyTutor pourra appeler : `search_python_docs` (recherche dans la doc Python via le RAG de la Phase 2), `run_python_code` (exécute du code en `subprocess`), `generate_exercise` (utilise la chaîne d'explication de la Phase 1), et `check_pep8` (lance `pycodestyle` sur du code).

Chaque tool doit être décoré avec `@tool` de `langchain_core.tools`, et sa **docstring** doit décrire précisément à quoi il sert et *quand* l'appeler — c'est ce que le modèle va lire pour décider.

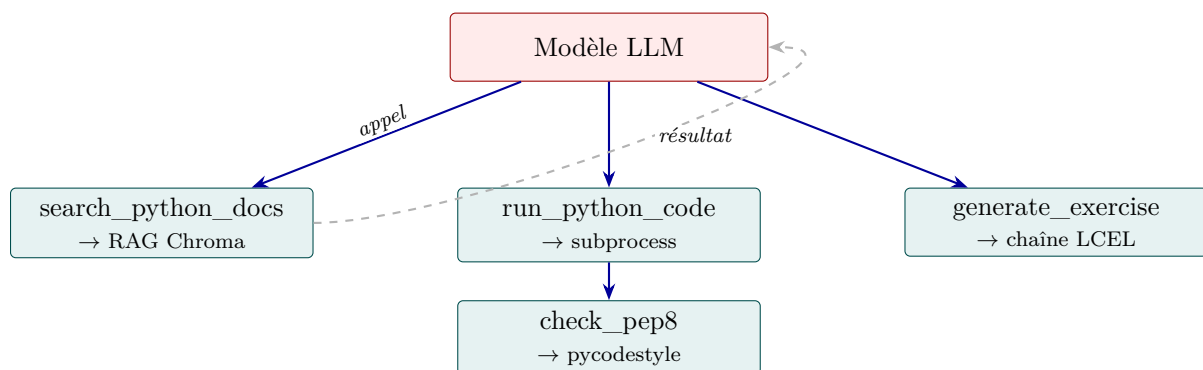
Contrainte : gérer proprement les cas d'erreur (timeout du subprocess, module absent, pas de résultat dans le RAG, valeur invalide en argument) avec des messages qui parlent au LLM.

Objectifs pédagogiques

- Maîtriser le décorateur `@tool` : il transforme une fonction Python en *tool* callable par un LLM.
- Comprendre que la **docstring** est envoyée au modèle comme description du tool — donc c'est du *prompt engineering* appliqué à du code.
- Apprendre les patterns d'implémentation propres : lazy init, `tempfile` avec nettoyage, `timeout`, fallbacks sur erreur.
- Réutiliser les briques des Phases 1 et 2 à l'intérieur d'un tool.

1.1 Solution

1.1.1 Architecture



Le LLM voit **4 tools déclarés** dans son prompt système (généré par `create_agent`). À chaque tour, il décide d'en appeler un (ou plusieurs) en lisant leur docstring. Le résultat de chaque appel retourne sous forme de `ToolMessage` dans l'historique, et le LLM *réfléchit* à nouveau.

1.1.2 Le code complet

```
1 import subprocess
2 import sys
3 import tempfile
4 from pathlib import Path
```

```

5
6 from langchain_core.tools import tool
7
8 from phase1_explain import explain_chain_for_level
9 from phase2_rag import build_or_load_vectorstore
10
11
12 # Cache lazy : on charge le retriever une seule fois, à la première utilisation
13 _retriever = None
14
15 def _get_retriever():
16     global _retriever
17     if _retriever is None:
18         vs = build_or_load_vectorstore()
19         _retriever = vs.as_retriever(search_kwargs={"k": 3})
20     return _retriever
21
22
23 @tool
24 def search_python_docs(query: str) -> str:
25     """Recherche dans la documentation Python officielle.
26
27     À utiliser pour toute question FACTUELLE sur Python : syntaxe d'une
28     construction, comportement standard d'une fonction built-in, vocabulaire
29     technique. PAS pour générer du code ou évaluer du code de l'élève.
30
31     Args:
32     query: La question ou les mots-clés à rechercher dans la documentation.
33     """
34     docs = _get_retriever().invoke(query)
35     if not docs:
36         return "Aucun résultat dans la doc Python pour cette recherche."
37     return "\n\n---\n\n".join(
38         f"Source: {d.metadata.get('source', '?')}\n{d.page_content}"
39         for d in docs
40     )
41
42
43 @tool
44 def run_python_code(code: str) -> str:
45     """Exécute du code Python dans un sous-processus et renvoie stdout + stderr.
46
47     À utiliser quand :
48     - l'élève montre un code qui ne marche pas (pour voir la vraie erreur)
49     - tu veux vérifier qu'un exemple que tu vas donner fonctionne réellement
50     - tu veux mesurer la sortie d'un bout de code pour pouvoir l'expliquer
51
52     Limites : timeout 5 secondes, environnement Python standard, pas de pip.
53
54     Args:
55     code: Le code Python à exécuter, complet et autonome.
56     """
57     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
58         f.write(code)
59         path = f.name
60     try:
61         result = subprocess.run(
62             [sys.executable, path],
63             capture_output=True, text=True, timeout=5,
64         )

```

```

65     parts = []
66     if result.stdout:
67         parts.append(f"STDOUT:\n{result.stdout}")
68     if result.stderr:
69         parts.append(f"STDERR:\n{result.stderr}")
70     parts.append(f"Exit code: {result.returncode}")
71     return "\n\n".join(parts)
72 except subprocess.TimeoutExpired:
73     return (
74         "ERREUR: timeout après 5 secondes. Le code boucle probablement "
75         "à l'infini (boucle while, récursion sans condition d'arrêt, etc.)"
76     )
77 finally:
78     Path(path).unlink(missing_ok=True)
79
80
81 @tool
82 def generate_exercise(topic: str, difficulty: str = "intermédiaire") -> str:
83     """Génère un exercice Python sur un concept avec ses questions de compréhension.
84
85     À utiliser quand l'élève demande explicitement de la pratique, des
86     exercices, ou pour vérifier sa compréhension d'un concept.
87
88     Args:
89         topic: Le concept Python à pratiquer (ex: "décorateurs", "list comprehensions").
90         difficulty: Le niveau visé. Valeurs valides : "débutant", "intermédiaire",
91             ↪ "avancé".
92     """
93     if difficulty not in {"débutant", "intermédiaire", "avancé"}:
94         difficulty = "intermédiaire"
95
96     chain = explain_chain_for_level(difficulty)
97     explanation = chain.invoke({"concept": topic})
98
99     questions = "\n\n".join(
100         f" Q{i}. {q.question}\n      -> Réponse attendue : {q.expected_answer}"
101         for i, q in enumerate(explanation.comprehension_questions, 1)
102     )
103     return (
104         f"Exercice sur '{topic}' (niveau {difficulty}) :\n\n"
105         f"Code de référence :\n{explanation.code_example}\n\n"
106         f"Questions :\n{questions}\n\n"
107         f"Piège classique : {explanation.pitfall}"
108     )
109
110 @tool
111 def check_pep8(code: str) -> str:
112     """Vérifie la conformité PEP 8 d'un bout de code Python.
113
114     À utiliser quand l'élève partage du code et veut savoir s'il est
115     propre, ou quand tu donnes un exemple que tu veux valider stylistiquement.
116
117     Args:
118         code: Le code Python à analyser.
119     """
120     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
121         f.write(code)
122         path = f.name
123     try:

```

```

124     result = subprocess.run(
125         [sys.executable, "-m", "pycodestyle", "--max-line-length=100", path],
126         capture_output=True, text=True, timeout=5,
127     )
128     if not result.stdout.strip():
129         return "Aucune violation PEP 8 détectée."
130     cleaned = result.stdout.replace(path, "<code>")
131     return f"Problèmes PEP 8 détectés :\n{cleaned}"
132 except FileNotFoundError:
133     return (
134         "Le module pycodestyle n'est pas installé. "
135         "Réponds à l'élève en lui suggérant `pip install pycodestyle`."
136     )
137 finally:
138     Path(path).unlink(missing_ok=True)

```

1.1.3 Explications pas à pas

Étape 1 — Le décorateur @tool

@tool de langchain_core.tools transforme une fonction Python en BaseTool consommable par un LLM. Ce qu'il fait sous le capot :

- Lit la **signature** (noms et types des arguments) pour générer un schéma JSON.
- Lit la **docstring** pour produire la *description* du tool.
- Encapsule l'appel pour intercepter exceptions et timeouts (à compléter manuellement pour les cas avancés).

```

1 @tool
2 def my_tool(query: str) -> str:
3     """Description du tool, en français OK."""
4     return "résultat"
5
6 # my_tool est maintenant un BaseTool, pas une fonction normale
7 my_tool.name          # "my_tool"
8 my_tool.description    # "Description du tool, en français OK."
9 my_tool.args_schema    # Pydantic model avec query: str

```

Étape 2 — La docstring est le contrat avec le LLM

La docstring est **envoyée au modèle** comme description du tool. Le modèle décide quand l'appeler en lisant cette description. Donc :

- **À QUOI** sert le tool — première ligne, factuelle.
- **QUAND** l'utiliser — situations explicites.
- **QUAND PAS** l'utiliser — démarquage des autres tools.
- **Args** typés et décrits — pour que le modèle remplisse bien chaque paramètre.

Une docstring vague = un agent qui prend de mauvaises décisions. Pour chaque tool, demande-toi : « si je donne CETTE docstring à un humain, peut-il savoir QUAND l'appeler sans hésitation ? » Si non, précise.

Étape 3 — Les paramètres typés

Les types Python (`str`, `int`, `list[str]`, `Literal[...]`...) sont traduits en JSON Schema dans la signature envoyée au LLM. Le modèle ne pourra pas appeler le tool avec le mauvais type — c'est validé par Pydantic en amont.

Pour `generate_exercise`, on accepte un `difficulty: str` au lieu d'un `Literal` parce qu'on veut un fallback côté Python plutôt qu'un crash si le modèle invente un niveau `"expert"`. C'est un choix : rigide via `Literal` pour échouer fort, ou souple via `str + validation` pour dégrader proprement.

Étape 4 — Patterns d'implémentation

Lazy initialisation. Le retriever du RAG (Phase 2) est lourd à charger. On le *cache* dans une variable de module avec `global` :

```
1 _retriever = None
2
3 def _get_retriever():
4     global _retriever
5     if _retriever is None:
6         vs = build_or_load_vectorstore()
7         _retriever = vs.as_retriever(search_kwargs={"k": 3})
8     return _retriever
```

Le retriever ne se charge que si le tool est effectivement appelé, et seulement la première fois.

Tempfile avec nettoyage. `tempfile.NamedTemporaryFile(delete=False)` + bloc `try/finally` qui supprime explicitement :

```
1 with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
2     f.write(code)
3     path = f.name
4 try:
5     # ... utilise path ...
6     pass
7 finally:
8     Path(path).unlink(missing_ok=True)
```

Timeout sur subprocess. Une boucle `while True` dans le code utilisateur bloquerait l'agent indéfiniment. Le `timeout=5` de `subprocess.run` lève `TimeoutExpired` :

```
1 try:
2     result = subprocess.run(..., timeout=5)
3 except subprocess.TimeoutExpired:
4     return "ERREUR: timeout après 5 secondes. Le code boucle..."
```

Messages d'erreur qui parlent au LLM. Quand un tool échoue, le message de retour devient l'observation que voit le modèle. Mauvais : `"Error 127"`. Bon : `"Le module pycodestyle n'est pas installé. Réponds à l'élève en lui suggérant pip install pycodestyle."` — le LLM sait alors quoi répondre.

Un tool n'a pas le droit de planter silencieusement, ni de lever une exception non gérée : dans les deux cas, l'agent perd la main et l'utilisateur voit un crash. Toujours renvoyer un message clair en cas d'erreur, même un message d'erreur.

Étape 5 — Réutiliser les briques des phases précédentes

Chaque tool s'appuie sur ce qu'on a construit précédemment :

Tool	Brique réutilisée
search_python_docs	Vector store + retriever (Phase 2).
run_python_code	subprocess standard, pas LangChain.
generate_exercise	Chaîne <code>explain_chain_for_level</code> avec <code>with_structured_output</code> (Phase 1).
check_pep8	pycodestyle en subprocess.

Un tool peut contenir une chaîne LCEL complète, voire un autre RAG. C'est le pattern « tool qui appelle un autre LLM en interne ». Très puissant pour décomposer un système complexe en agents qui se passent des tâches.

1.2 Vérification dans LangSmith

Quand un tool est appelé par l'agent, LangSmith trace :

```
ToolNode (search_python_docs)
  Input : {"query": "Comment fonctionne yield ?"}
  Output : "Source: docs.python.org/.../functional.html..."
ToolNode (run_python_code)
  Input : {"code": "def f():..."}
  Output : "STDOUT: ... | Exit code: 0"
```

Si un tool ne s'appelle jamais alors qu'il devrait : regarde sa docstring. Si un tool s'appelle trop : regarde aussi sa docstring. C'est presque toujours là.

Récapitulatif

Ce qu'il faut retenir :

1. **@tool** transforme une fonction Python en outil consommable par un LLM. Signature et docstring deviennent un contrat envoyé au modèle.
2. La docstring fait du **prompt engineering** sur le code : explicite à **quoi** sert le tool, **quand** l'appeler, et **quand pas**.
3. Patterns essentiels : **lazy init** pour les ressources lourdes, **tempfile** avec nettoyage, **timeout** sur les subprocess, messages d'erreur qui parlent au LLM.
4. Un tool peut contenir une chaîne LCEL, un RAG, ou n'importe quel autre traitement Python — c'est l'orchestration côté agent qui le rend pertinent.

2 Construire l'agent avec `create_agent`

Exercice 2. Assembler les tools dans un agent ReAct

Construis un agent qui combine : le modèle de chat, la liste des 4 tools définis dans l'exercice 1, et un `system_prompt` qui dicte le comportement attendu (français, pédagogique, règles d'usage des tools).

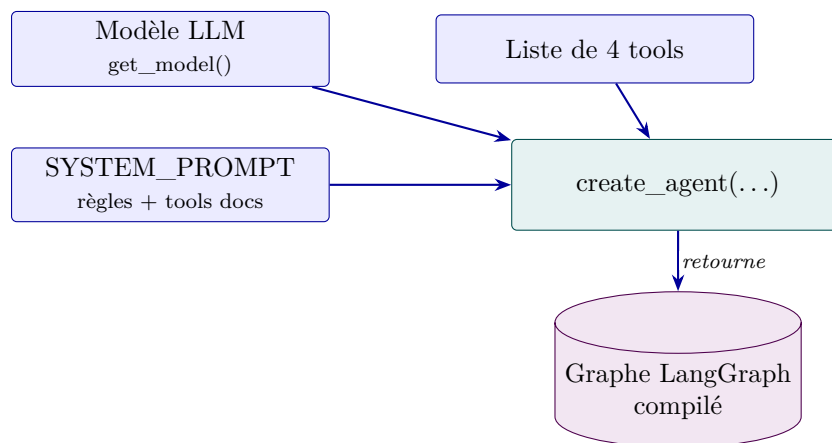
Contrainte : utiliser `create_agent` de `langchain.agents` (LangChain 1.x), qui renvoie un *graphe LangGraph* compilé prêt à `.invoke` ou `.stream`.

Objectifs pédagogiques

- Maîtriser `create_agent(model, tools, system_prompt)` : la voie officielle LangChain 1.x pour les agents ReAct.
- Rédiger un `system_prompt` qui couvre rôle, périmètre, règles de comportement, et choix de langue.
- Comprendre la signature d'invocation : `agent.invoke({"messages": [...]})`.
- Savoir que le retour de `create_agent` est un **graphe LangGraph** (donc tout LangGraph s'applique).

2.1 Solution

2.1.1 Architecture



`create_agent` construit le graphe d'exécution de la boucle ReAct : un nœud LLM qui décide, un nœud `ToolNode` qui exécute, et des arêtes conditionnelles qui les enchaînent jusqu'à la réponse finale.

2.1.2 Le code complet

```
1 from langchain.agents import create_agent
2 from config import get_model
3
4
5 TOOLS = [search_python_docs, run_python_code, generate_exercise, check_pep8]
6
7 SYSTEM_PROMPT = """Tu es PyTutor, un assistant pédagogique pour des élèves apprenant
8   ↪ Python.
9
10 Tu as accès à 4 outils :
11 - search_python_docs : pour récupérer du contenu de la doc Python officielle
12 - run_python_code    : pour exécuter du code et observer ce qui se passe
13 - generate_exercise  : pour générer un exercice et ses questions
```

```

13 - check_pep8          : pour vérifier le style PEP 8
14
15 Règles de comportement :
16 1. Pour toute question factuelle sur Python (syntaxe, builtin, vocabulaire),
17    utilise search_python_docs AVANT de répondre. Cite la source dans ta réponse.
18 2. Quand l'élève montre un code qui ne marche pas, exécute-le toi-même avec
19    run_python_code AVANT d'expliquer pourquoi, pour voir la vraie erreur.
20 3. Quand tu donnes un exemple de code dans ta réponse, exécute-le d'abord pour
21    être sûr qu'il marche.
22 4. Sois pédagogique : explique POURQUOI, pas seulement COMMENT.
23 5. Adapte ton vocabulaire au niveau de l'élève (dédus-le de ses questions).
24 6. Réponds en français.
25 """
26
27
28 def build_agent():
29     """Construit l'agent PyTutor avec ses 4 tools."""
30     return create_agent(
31         model=get_model(),
32         tools=TOOLS,
33         system_prompt=SYSTEM_PROMPT,
34     )

```

2.1.3 Explications pas à pas

Étape 1 — La signature de create_agent

```

1 create_agent(
2     model,                # instance ChatModel ou string "provider:model"
3     tools,                # liste de @tool callables
4     system_prompt=None,   # str, remplace l'ancien `prompt`
5     # ... d'autres options avancées
6 )

```

Trois paramètres essentiels :

- **model** : une instance issue de `get_model()` (ou n'importe quel `BaseChatModel`). Ou une string courte "anthropic:claude-3-5-sonnet-20241022".
- **tools** : la liste de fonctions décorées `@tool`. L'ordre n'a pas d'importance fonctionnelle, mais peut influencer marginalement les décisions du modèle (priorité au premier tool listé en cas d'ambiguïté).
- **system_prompt** : instructions générales. C'est le *contrat global* de l'agent — son rôle, ses règles, son ton.

Étape 2 — Anatomie d'un system_prompt efficace

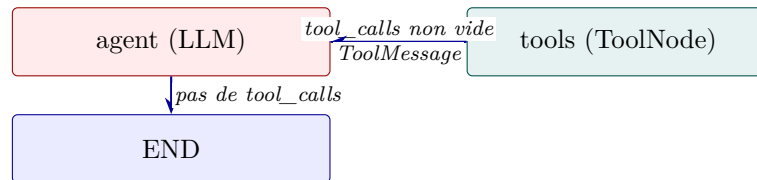
Le SYSTEM_PROMPT de PyTutor a quatre sections distinctes :

1. **Identité** (« Tu es PyTutor, un assistant pédagogique ... »).
2. **Inventaire des tools** (la même liste qu'aux yeux du modèle, en clair). Redondant mais utile : ancre la décision.
3. **Règles d'usage** numérotées qui couvrent les cas critiques (utilise X avant Y, exécute le code avant de l'expliquer, etc.).
4. **Méta-comportement** (langue, ton, public).

*Le `system_prompt` n'est pas un texte décoratif : c'est le **principal levier de qualité** d'un agent. Une règle bien formulée évite des centaines d'appels de tool inutiles. Investis-y du temps.*

Étape 3 — Le graphe LangGraph sous le capot

`create_agent` retourne un **graphe LangGraph compilé**, pas un simple `Runnable`. Visuellement, ce graphe a la forme :



À chaque tour, le nœud LLM examine la conversation. Soit il décide d'appeler des tools (le graphe passe au `ToolNode`, exécute les appels, revient au LLM), soit il produit une réponse finale (le graphe se termine à `END`).

Étape 4 — Invocation : `{"messages": [...]}`

L'agent s'utilise avec un dict contenant la clé `messages` :

```
1 from langchain_core.messages import HumanMessage
2
3 agent = build_agent()
4 result = agent.invoke({
5     "messages": [HumanMessage(content="Comment fonctionne yield ?")]
6 })
7
8 # result["messages"] contient TOUT l'historique :
9 # [SystemMessage, HumanMessage, AIMessage(tool_calls=...), ToolMessage, AIMessage]
```

Le `system_prompt` est injecté automatiquement au début par `create_agent` : tu n'as pas à le passer toi-même dans `messages`.

Étape 5 — Le retour est l'historique complet

`result["messages"]` contient **toute la conversation**, pas seulement la dernière réponse :

- Le `SystemMessage` (ajouté par `create_agent`).
- Le `HumanMessage` initial.
- Les `AIMessage` intermédiaires, parfois avec `tool_calls`.
- Les `ToolMessage` qui contiennent les résultats des tools.
- Le `AIMessage` final (la réponse à l'utilisateur).

C'est cette structure qu'on va lire dans les démos de l'exercice 3.

2.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```
LangGraph (agent compilé)
  agent (LLM) ← tour 1
  tools (ToolNode) ← exécute le(s) appel(s)
  agent (LLM) ← tour 2
  (éventuellement tools → agent à nouveau)
END
```

Trois choses à regarder dans une trace d'agent :

1. Le **system_prompt effectif** dans le premier appel LLM (onglet *Input*). Vérifie qu'il contient tes règles ET les descriptions des tools.
2. Les **tool_calls** dans chaque AIMessage : nom du tool appelé, arguments choisis par le LLM.
3. Les **ToolMessage** : ce que les tools ont *vraiment* renvoyé au LLM. Une réponse incorrecte vient souvent d'un tool qui a renvoyé un truc inattendu.

Récapitulatif

Ce qu'il faut retenir :

1. *`create_agent(model, tools, system_prompt)` construit un agent ReAct prêt à l'emploi en LangChain 1.x.*
2. *Le retour est un **graphe LangGraph compilé** : tout LangGraph s'applique (state, persistance, streaming, etc.).*
3. *Le **system_prompt** doit avoir 4 parties : identité, inventaire des tools, règles numérotées, méta-comportement.*
4. *Invocation : `agent.invoke({"messages": [HumanMessage(...)]})`.*
5. *Le retour est l'historique **complet** de la conversation (System / Human / AI / Tool / AI / ...).*

3 Observer la boucle Réfléchir-Agir-Observer (ReAct)

Exercice 3. Trois démos contrastées pour voir la boucle ReAct

Lance l'agent sur trois requêtes qui déclenchent des comportements différents :

1. **Question factuelle** : doit déclencher `search_python_docs`.
2. **Code qui boucle** : doit déclencher `run_python_code` (l'agent voit le timeout en clair).
3. **Question multi-étape** : doit déclencher *plusieurs* tools en chaîne (recherche + exercice).

Écris une fonction `print_trace(messages)` qui formate l'historique de la conversation pour qu'on *voie* le déroulé de la boucle ReAct.

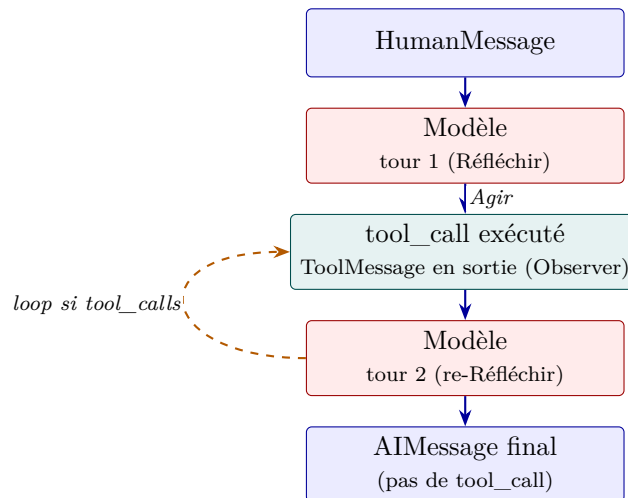
Contrainte : distinguer visuellement `HumanMessage`, `AIMessage` (texte vs `tool_calls`), et `ToolMessage` (résultat des tools).

Objectifs pédagogiques

- Lire l'historique de messages d'un agent : ordre, types, contenu.
- Comprendre la boucle **Réfléchir-Agir-Observer** : le LLM décide → le tool s'exécute → le LLM observe → il re-décide jusqu'à pouvoir répondre.
- Identifier quand un agent décide de *s'arrêter* : l'`AIMessage` sans `tool_calls` est la réponse finale.
- Construire un affichage lisible pour le déroulé pédagogique.

3.1 Solution

3.1.1 Architecture



À chaque tour :

1. Le LLM *réfléchit* : examine la conversation et décide.
2. S'il décide d'appeler un ou plusieurs tools, il les liste dans `AIMessage.tool_calls`.
3. Le `ToolNode agit` : exécute chaque appel et produit un `ToolMessage` par tool.
4. Le LLM *observe* : ces `ToolMessage` entrent dans son contexte au tour suivant.
5. Boucle jusqu'à ce que le LLM produise un `AIMessage` sans `tool_calls` — c'est la réponse finale.

3.1.2 Le code complet

```
1 from langchain_core.messages import (  
2     HumanMessage, AIMessage, ToolMessage, SystemMessage,
```

```

3 )
4
5
6 def print_trace(messages, from_index: int = 0) -> None:
7     """Affiche le déroulé de la boucle ReAct depuis un index donné."""
8     for msg in messages[from_index:]:
9         if isinstance(msg, SystemMessage):
10             continue # on n'affiche pas le system prompt
11         elif isinstance(msg, HumanMessage):
12             print(f"\n[USER]:\n {msg.content}")
13         elif isinstance(msg, AIMessage):
14             if msg.content:
15                 print(f"\n[AI]:\n {msg.content}")
16             if hasattr(msg, "tool_calls") and msg.tool_calls:
17                 print(f"\n[AI] décide d'appeler {len(msg.tool_calls)} tool(s) :")
18                 for tc in msg.tool_calls:
19                     args_preview = str(tc["args"])[0:100]
20                     suffix = "... " if len(str(tc["args"])) > 100 else ""
21                     print(f"    -> {tc['name']}({args_preview}{suffix})")
22             elif isinstance(msg, ToolMessage):
23                 preview = msg.content[:300].replace("\n", "\n    ")
24                 ellipsis = "... " if len(msg.content) > 300 else ""
25                 print(f"\n[TOOL {msg.name}]:\n    {preview}{ellipsis}")
26
27
28 def demo_factual_question() -> None:
29     """Question factuelle : devrait déclencher search_python_docs."""
30     agent = build_agent()
31     result = agent.invoke({
32         "messages": [HumanMessage(content="À quoi sert exactement le mot-clé `yield` ?")]
33     })
34     print_trace(result["messages"])
35
36
37 def demo_buggy_code() -> None:
38     """Code qui boucle à l'infini : devrait déclencher run_python_code."""
39     buggy = """\
40 def double_each(numbers):
41     for n in numbers:
42         numbers.append(n * 2)
43     return numbers
44
45 print(double_each([1, 2, 3]))
46 """
47     agent = build_agent()
48     result = agent.invoke({
49         "messages": [HumanMessage(content=(
50             "Mon code ne se termine jamais. Tu peux m'aider ?\n\n"
51             f"```python\n{buggy}```"
52         ))]
53     })
54     print_trace(result["messages"])
55
56
57 def demo_multi_tool() -> None:
58     """Question complexe : devrait chaîner plusieurs tools."""
59     agent = build_agent()
60     result = agent.invoke({
61         "messages": [HumanMessage(content=(
62             "Explique-moi les list comprehensions en t'appuyant sur la doc, "

```

```

63         "puis génère-moi un exercice de niveau intermédiaire pour pratiquer."
64     )))
65 }
66 print_trace(result["messages"])

```

3.1.3 Explications pas à pas

Étape 1 — La structure de l'historique de messages

`result["messages"]` est une liste hétérogène, dans l'ordre chronologique :

Type	Rôle
SystemMessage	Le <code>system_prompt</code> injecté par <code>create_agent</code> . Toujours en première position.
HumanMessage	Une requête utilisateur.
AIMessage	Une réponse (partielle ou finale) du LLM. Peut contenir : <code>content</code> (texte), <code>tool_calls</code> (liste d'appels à exécuter), ou les deux.
ToolMessage	Le résultat d'un appel de tool. Contient <code>name</code> (nom du tool), <code>content</code> (résultat str), <code>tool_call_id</code> (corrélation avec l' <code>AIMessage.tool_calls</code>).

Étape 2 — La boucle ReAct en pratique

Pour la démo *question factuelle*, on observe typiquement :

1. HumanMessage : « À quoi sert yield ? »
2. AIMessage (tour 1, **Réfléchir**) : pas de `content`, juste `tool_calls = [search_python_docs(query="yield")]`
3. ToolMessage (**Agir + Observer**) : contient les chunks de la doc sur yield.
4. AIMessage (tour 2, **re-Réfléchir**) : `content` = réponse finale, pas de `tool_calls`.

C'est une boucle ReAct à **une seule itération** : l'agent réfléchit, agit une fois, observe, puis répond.

Pour la démo *multi-tool*, la boucle a typiquement 2 itérations : l'agent appelle d'abord `search_python_docs` pour la doc, *observe* le résultat, puis appelle `generate_exercise` pour l'exercice, observe à nouveau, puis répond.

Étape 3 — La condition d'arrêt

L'agent s'arrête **exactement** quand le LLM produit un `AIMessage` sans `tool_calls`. Ce n'est pas un nombre maximal de tours imposé par défaut : c'est le LLM qui décide qu'il a assez d'informations pour répondre.

***Garde-fou de sécurité** : par défaut, le graphe a un `recursion_limit` (typiquement 25 tours) qui force l'arrêt si l'agent boucle sans converger. C'est rare en pratique mais utile pour détecter un bug de prompt (« utilise toujours `search_python_docs` » qui crée une boucle infinie de recherches).*

Étape 4 — `print_trace` : afficher proprement

Trois choix d'affichage :

- Distinguer **visuellement** chaque type de message avec un préfixe (`[USER]`, `[AI]`, `[TOOL nom]`). En production, on utiliserait des emojis ou des couleurs.

- **Tronquer** les contenus longs ([:300]) pour garder un déroulé lisible. Sans troncature, un ToolMessage avec 4000 caractères de doc rend la trace illisible.
- **Indenter** les contenus pour qu'on voit la structure d'un coup d'œil.

Sortie de l'exécution

```
=====
DEMO 1 -- Question factuelle
=====

[OK] Rechargement de l'index depuis ./chroma_db

[USER]:
  A quoi sert exactement le mot-clé `yield` ?

[AI]:
  Je vais chercher la documentation officielle pour te donner une
  explication précise.
  -> tool_use: search_python_docs({'query': 'yield keyword
                                   generator'})

[TOOL search_python_docs]:
  Source: https://docs.python.org/3/howto/functional.html
  are messy. In Python 2.5 there's a simple way to pass values
  into a generator. yield became an expression, returning a
  value that can be assigned to a variable or otherwise operated
  on: val = (yield i) ...

[AI]:
  Parfait ! Voici l'explication :

  ## Le mot-clé `yield`
  **`yield` sert à créer des générateurs** -- des fonctions qui
  produisent une séquence de valeurs **une à la fois**, sans tout
  charger en mémoire d'un coup.

  ### Différence clé avec `return`
  - **`return`** : termine la fonction et retourne une valeur
  - **`yield`** : **suspend** la fonction, retourne une valeur, et
    **mémorise l'état** pour reprendre plus tard

  -> tool_use: run_python_code({...comparaison classique/yield...})

[TOOL run_python_code]:
  STDOUT:
  === Approche classique ===
  Type : <class 'list'>
  Contenu : [0, 1, 2, 3, 4]

  === Approche avec yield ===
  Type : <class 'generator'>
  Contenu : <generator object nombres_generateur at 0x1029af780>
  Valeurs du générateur :
  0
  1
  2
  3
  4
  Exit code: 0

[AI]:
  ### Avantages des générateurs
  1. **Economie de mémoire** : les valeurs sont générées à la
  demande, pas stockées
  2. **Paresse** (lazy evaluation) : utile pour les grandes
```



```

séquences ou infinies
3. **Lisibilité** : code plus simple et naturel

-> tool_use: run_python_code({...comparaison taille mémoire...})

[TOOL run_python_code]:
  STDOUT:
  Taille d'une liste de 1M nombres : 8000056 bytes
  Taille d'un générateur : 192 bytes
  Exit code: 0

[AI]:
  Enorme différence !

  ### Résumé
  | Aspect          | `return`          | `yield`          |
  |-----|-----|-----|
  | Retourne        | Une valeur unique | Un générateur    |
  | Mémoire         | Tout en mémoire  | A la demande     |
  | Reprendre ?    | Non, terminée    | Oui, état préservé |
  | Cas d'usage    | Résultat unique  | Séquences        |

  Source : Documentation Python officielle - Generators

=====
DEMO 2 -- Debugging d'un code qui boucle
=====

[USER]:
  Mon code ne se termine jamais. Tu peux m'aider ?

  ```python
 def double_each(numbers):
 for n in numbers:
 numbers.append(n * 2)
 return numbers

 print(double_each([1, 2, 3]))
  ```

[AI]:
  Je vais exécuter ton code pour voir ce qui se passe :
  -> tool_use: run_python_code({'code': '...'})

[TOOL run_python_code]:
  ERREUR: timeout après 5 secondes. Le code boucle probablement
  à l'infini (boucle while, génération récursive sans condition
  d'arrêt, etc.)

[AI]:
  Exactement ! Ton code crée une **boucle infinie**. Voici pourquoi :

  ## Le problème
  Tu modifies la liste **pendant que tu la parcoures** :
  Iteration 1 : n=1, append(2) -> [1, 2, 3, 2]
  Iteration 2 : n=2, append(4) -> [1, 2, 3, 2, 4]
  Iteration 3 : n=3, append(6) -> [1, 2, 3, 2, 4, 6]
  Iteration 4 : n=2, append(4) -> [1, 2, 3, 2, 4, 6, 4] <- continue !

  ## La solution
  **Option 1 : Créer une nouvelle liste**
  ```python
 def double_each(numbers):
 result = []

```

```

 for n in numbers:
 result.append(n * 2)
 return result
 ...

Option 2 : Parcourir une copie
```python
def double_each(numbers):
    for n in numbers[:]: #[:] crée une copie
        numbers.append(n * 2)
    return numbers
...

**Option 3 : List comprehension (plus pythonique)**
```python
def double_each(numbers):
 return [n * 2 for n in numbers]
...

```

=====

DEMO 3 -- Multi-tool (explication + exercice)

=====

[USER]:

Explique-moi les list comprehensions en t'appuyant sur la doc,  
puis génère-moi un exercice de niveau intermédiaire pour pratiquer.

[AI]:

Je vais d'abord consulter la documentation officielle sur les  
list comprehensions, puis je te générerai un exercice.

```

-> tool_use: search_python_docs({'query': 'list comprehensions'})
-> tool_use: generate_exercise({'topic': 'list comprehensions',
 'difficulty': 'intermédiaire'})

```

[TOOL search\_python\_docs]:

Source: <https://docs.python.org/3/tutorial/datastructures.html>  
For example, assume we want to create a list of squares, like:

```

>>> squares = []
>>> for x in range(10):
... squares.append(x ** 2)
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```

Note that this creates (or overwrites) a vari...

[TOOL generate\_exercise]:

Exercice sur 'list comprehensions' (niveau intermédiaire) :

Code de référence :

```

Approche classique avec une boucle
nombres = [1, 2, 3, 4, 5]
carres_classique = []
for n in nombres:
 carres_classique.append(n ** 2)
Approche avec list comprehension
carres = [n ** 2 for n in...]

```

[AI]:

## Les List Comprehensions expliquées

Une **list comprehension** est une syntaxe compacte pour créer une  
liste. Selon la doc, elle se compose de :

- Des **crochets** `[ ]`
- Une **expression** à évaluer
- Une ou plusieurs **clauses** `for`
- Zéro ou plusieurs **clauses** `if` (optionnelles)

```

Syntaxe générale :
```python
[expression for variable in iterable if condition]
```

Exemples clés
1. Avec une condition (filtrage) :
```python
[x for x in range(10) if x % 2 == 0]    # [0, 2, 4, 6, 8]
```

2. Avec plusieurs boucles (produit cartésien) :
```python
[(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
```

Exercice de niveau intermédiaire
Q1. Qu'est-ce qu'une list comprehension et quel est son
avantage par rapport à une boucle for classique ?

Q2. Ecris une list comprehension qui crée une liste contenant
les carrés des nombres pairs de 1 à 10.

Q3. Pourquoi serait-il mauvais d'utiliser une list comprehension
pour une transformation très complexe impliquant plusieurs
conditions imbriquées et appels de fonctions ?

Piège classique à éviter :
- Confondre l'ordre : `[for x in liste expression]` (FAUX)
- Bon ordre : `[expression for x in liste]` (CORRECT)

```

## 3.2 Vérification dans LangSmith

LangSmith trace chaque agent run avec une visualisation arborescente :

```

LangGraph run
agent (LLM, tour 1)
Input : [System, Human]
Output : tool_calls = [{search_python_docs}]
tools (ToolNode)
search_python_docs(query="yield")
Output : "Source: ..."
agent (LLM, tour 2)
Input : [System, Human, AI, Tool]
Output : "Le mot-clé yield..."

```

*La vue timeline dans LangSmith montre la durée de chaque tour : on voit instantanément si un tool est lent (souvent `search_python_docs` avec `embedding` + recherche), si le LLM est lent (gros prompt), ou si la boucle a trop d'itérations (probablement un bug de prompt).*

## Récapitulatif

*Ce qu'il faut retenir :*

1. L'historique d'un agent est une liste hétérogène : *System*, *Human*, *AIMessage* (avec ou sans *tool\_calls*), *ToolMessage*.
2. Boucle ReAct : **Réfléchir** (LLM décide) → **Agir** (tools exécutent) → **Observer** (LLM relit) → re-Réfléchir, jusqu'à un *AIMessage* sans *tool\_calls*.
3. L'agent décide seul de s'arrêter : il n'y a pas de nombre fixe de tours, juste un garde-fou *recursion\_limit* en cas de bug.
4. Pour debugger un agent : **lit l'historique**, regarde les *tool\_calls* et les *ToolMessage*. La plupart des problèmes sont là.

## 4 Mémoire conversationnelle multi-tours

### Exercice 4. Un chat où l'agent se souvient des tours précédents

Construis une boucle de chat (*REPL*) où l'utilisateur peut poser plusieurs questions successives, et où l'agent **se souvient** des tours précédents (utile pour les « peux-tu approfondir ? », « et avec un exemple ? »...).

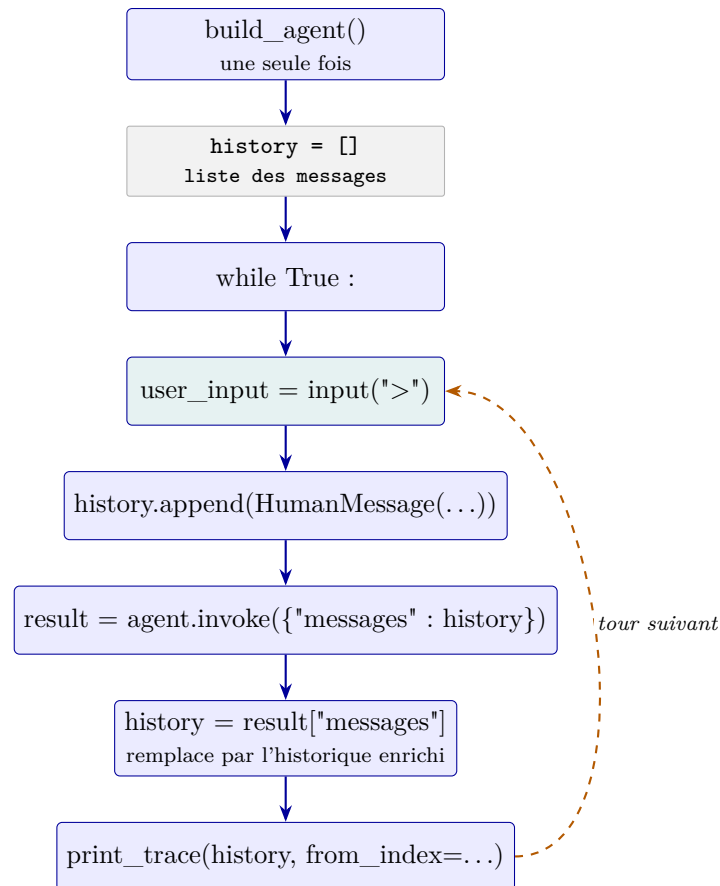
**Contrainte** : `create_agent` ne stocke *rien* entre les invocations. La mémoire est à maintenir côté client en accumulant l'historique de `messages` et en le réinjectant à chaque tour.

#### Objectifs pédagogiques

- Comprendre que l'agent par défaut est **stateless** : chaque `invoke` est indépendant.
- Construire la mémoire côté client en accumulant la liste de messages.
- Quantifier le coût (en tokens) d'une mémoire qui s'allonge à chaque tour.
- Connaître les alternatives (`checkpoint` LangGraph) pour une persistance native côté serveur.

## 4.1 Solution

### 4.1.1 Architecture



**Important :** l'agent est construit une seule fois (hors boucle). À chaque tour, on accumule un nouveau `HumanMessage`, on `invoke` avec *tout* l'historique, et on remplace l'historique par celui que renvoie l'agent (qui contient maintenant les nouveaux `AIMessage` et `ToolMessage`).

### 4.1.2 Le code complet

```
1 def chat_loop() -> None:
2 """Boucle de chat interactive avec mémoire conversationnelle.
3
4 create_agent ne stocke RIEN entre les invocations. Pour avoir une
5 conversation continue, on accumule les messages côté client et on les
6 renvoie à chaque tour.
7 """
8 print("MODE CHAT - Ctrl+C ou ligne vide pour quitter")
9
10 agent = build_agent()
11 history: list = [] # accumulera HumanMessage / AIMessage / ToolMessage
12
13 while True:
14 try:
15 user_input = input("\n> ").strip()
16 except (EOFError, KeyboardInterrupt):
17 print("\nÀ bientôt.")
18 return
19
20 if not user_input:
21 print("Au revoir.")
```

```

22 return
23
24 # 1. On ajoute le nouveau message au contexte complet
25 history.append(HumanMessage(content=user_input))
26 index_before = len(history) - 1
27
28 # 2. L'agent renvoie l'historique complet enrichi
29 result = agent.invoke({"messages": history})
30 history = result["messages"]
31
32 # 3. On n'affiche que les messages produits ce tour-ci
33 print_trace(history, from_index=index_before)

```

### 4.1.3 Explications pas à pas

#### Étape 1 — Pourquoi l'agent est stateless par défaut

`create_agent` retourne un graphe `LangGraph` compilé qui, par défaut, n'a **pas de checkpointer**. Chaque `.invoke()` exécute le graphe *from scratch* avec uniquement les messages qu'on lui donne en entrée. Aucune mémoire entre appels.

C'est un choix de design : la persistance est un *problème séparé* (où stocker, comment indexer, comment expirer). `LangChain` ne veut pas imposer un backend particulier.

#### Étape 2 — Maintenir l'historique côté client

Le pattern est simple : on garde une liste `history`, on y ajoute le nouveau message utilisateur, on `invoke`, et on remplace `history` par ce que l'agent renvoie :

```

1 history.append(HumanMessage(content=user_input))
2 result = agent.invoke({"messages": history})
3 history = result["messages"]

```

Après `invoke`, `history` contient :

- Tout l'historique précédent.
- Le nouveau `HumanMessage`.
- Les éventuels `AIMessage(tool_calls=...)` intermédiaires.
- Les éventuels `ToolMessage` avec leurs résultats.
- Le `AIMessage` final.

Au tour suivant, ces messages sont *tous* renvoyés à l'agent — le LLM peut donc se référer aux échanges précédents.

#### Étape 3 — Le coût en tokens augmente à chaque tour

| Tour | Tokens d'entrée du LLM            | Cumul |
|------|-----------------------------------|-------|
| 1    | system + question 1 (~300 tokens) | 300   |
| 2    | + AI 1 + Human 2 (~800 tokens)    | 1100  |
| 3    | + AI 2 + Human 3 (~1500 tokens)   | 2600  |
| ...  | ...                               | ...   |
| 10   | cumul ~15000 tokens               | 15000 |

À chaque tour, on *paie* l'intégralité de l'historique précédent en tokens d'entrée. Sur des conversations longues, ça devient coûteux.

***Mitigation** : tronquer l'historique (garder les  $N$  derniers messages), ou résumer les anciens messages avec une chaîne LCEL quand la conversation dépasse une certaine taille (technique de la summary memory).*

## Étape 4 — Termination propre

Trois façons de sortir du chat :

- `Ctrl+C` : `input()` lève `KeyboardInterrupt`.
- `Ctrl+D` : `input()` lève `EOFError`.
- **Ligne vide** : `input()` renvoie "", on traite explicitement.

## Étape 5 — Alternative serveur : le checkpointer LangGraph

Pour une persistance native (utile dans un backend web), LangGraph fournit des *checkpointers* :

```
1 from langgraph.checkpoint.memory import MemorySaver
2
3 agent = create_agent(
4 model=get_model(),
5 tools=TOOLS,
6 system_prompt=SYSTEM_PROMPT,
7 checkpointer=MemorySaver(), # ou SqliteSaver, PostgresSaver, ...
8)
9
10 # Chaque conversation est identifiée par un thread_id :
11 config = {"configurable": {"thread_id": "user-42"}}
12 result = agent.invoke(
13 {"messages": [HumanMessage(content="Salut")]},
14 config=config,
15)
```

Le checkpointer stocke l'état du graphe (donc l'historique) entre les appels. L'utilisateur n'a plus à maintenir `history` côté client.

*Pattern de production : `MemorySaver` pour les tests, un `Sqlite`- ou `Postgres`-saver pour la prod. Avec un `thread_id` par session utilisateur, on a un chat multi-utilisateur sans aucune logique de mémoire côté client.*

## 4.2 Vérification dans LangSmith

Dans LangSmith, chaque `invoke` produit une **trace distincte** (sauf si on utilise un `trace` explicite). Les traces sont liées par leur projet et leur timestamp, mais pas automatiquement groupées par conversation.

Pour grouper les runs d'un même utilisateur :

- Avec checkpointer + `thread_id` : LangSmith expose le `thread_id` dans les métadonnées de chaque run, on peut filtrer.
- Sans checkpointer : passer manuellement un `run_name` ou des `tags` pour grouper.

*Astuce : dans la version stateless (notre exercice), tu peux comparer le nombre de tokens d'entrée entre les runs successifs pour visualiser la croissance de l'historique. Sur une conversation de 10 tours, c'est édifiant.*

## Récapitulatif

Ce qu'il faut retenir :

1. `create_agent` est **stateless** par défaut : chaque `.invoke()` est indépendant.
2. Pour une mémoire conversationnelle simple : accumuler les **messages** côté client, et réinjecter toute l'historique à chaque tour.
3. Le coût en tokens d'entrée **croît à chaque tour**. Sur conversations longues, tronquer ou résumer.
4. Pour une persistance native côté serveur : passer un **checkpoint** à `create_agent` et identifier chaque session par un `thread_id`.
5. Pattern REPL standard : `init` coûteux hors boucle, traitement rapide dans la boucle, terminaison propre sur `Ctrl+C` / `EOF` / ligne vide.

## Deuxième partie

# Pour aller plus loin

## 5 Ajouter un cinquième tool à l'agent

### Exercice bonus 1. Étendre l'inventaire de tools en une fonction

Écris un tool `current_python_version()` qui renvoie la version de Python utilisée par le processus, et ajoute-le à l'agent PyTutor. Vérifie ensuite avec une question genre « *Quelle version utilises-tu ? Est-ce que je peux utiliser `match/case` ?* ».

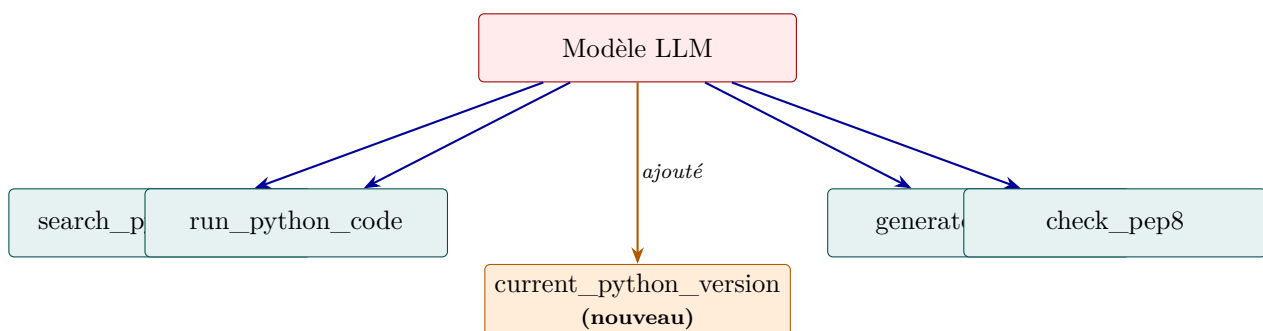
**Contrainte** : la docstring du tool doit mentionner les cas d'usage typiques (compatibilité, features par version : `match` en 3.10, `ExceptionGroup` en 3.11, etc.) pour que le LLM sache quand l'appeler.

#### Objectifs pédagogiques

- Mesurer la **trivialité** de l'ajout d'un tool une fois que l'agent est construit : une fonction `@tool`, et on l'ajoute à la liste.
- Comprendre que le modèle découvre le nouveau tool **automatiquement** via son nom et sa docstring — pas de configuration explicite.
- Voir le rôle des cas d'usage dans la docstring : c'est ce qui déclenche (ou pas) l'appel du tool selon la question.

### 5.1 Solution

#### 5.1.1 Architecture





Un cinquième tool s'ajoute sans modification du reste : même `system_prompt`, même `model`, même architecture LangGraph. Le modèle, à l'invocation, voit 5 tools dans son contrat au lieu de 4.

### 5.1.2 Le code complet

```
1 import sys
2 from langchain.agents import create_agent
3 from langchain_core.tools import tool
4 from langchain_core.messages import HumanMessage
5 from config import get_model
6 from phase3_agent import TOOLS, SYSTEM_PROMPT, print_trace
7
8
9 @tool
10 def current_python_version() -> str:
11 """Renvoie la version Python du processus qui exécute PyTutor.
12
13 À utiliser quand l'élève pose des questions sur la compatibilité, les
14 features récentes (match/case en 3.10, ExceptionGroup en 3.11, etc.),
15 ou simplement quand il demande quelle version est utilisée.
16 """
17 return f"Python {sys.version.split()[0]} ({sys.platform})"
18
19
20 def exercise_1() -> None:
21 # On rajoute simplement le nouveau tool à la liste existante.
22 agent = create_agent(
23 model=get_model(),
24 tools=TOOLS + [current_python_version],
25 system_prompt=SYSTEM_PROMPT,
26)
27
28 result = agent.invoke({
29 "messages": [HumanMessage(content=(
30 "Quelle version de Python utilises-tu pour exécuter mon code ? "
31 "Est-ce que je peux utiliser le pattern matching avec `match` / `case` ?"
32))]
33 })
34 print_trace(result["messages"])
```

### 5.1.3 Explications pas à pas

#### Étape 1 — La fonction Python décorée

Un tool n'a besoin que de trois choses : un décorateur `@tool`, une signature typée, une docstring :

```
1 @tool
2 def current_python_version() -> str:
3 """Renvoie la version Python..."""
4 return f"Python {sys.version.split()[0]} ({sys.platform})"
```

**Sans argument** : la signature `() -> str` est minimale. Le LLM n'a rien à fournir en entrée, juste à décider d'appeler le tool.

## Étape 2 — L'ajout à la liste tools

```
1 tools=TOOLS + [current_python_version]
```

C'est tout. Aucun re-enregistrement, aucune configuration. Le nouveau tool est passé à `create_agent` qui l'injecte dans le contrat du LLM.

***Pattern de composition** : importer la liste `TOOLS` de `phase3_agent.py` et la combiner avec de nouveaux tools évite de dupliquer la définition de l'agent. C'est l'équivalent d'une extension en orienté-objet, mais en plus simple : juste de la concaténation de listes.*

## Étape 3 — La docstring déclenche l'usage

La docstring liste des cas d'usage spécifiques :

- « questions sur la compatibilité » — déclenche l'appel quand l'utilisateur demande s'il peut utiliser `walrus` ou autre feature récente.
- « features récentes (match/case en 3.10...) » — donne au LLM des *exemples concrets* de questions qui doivent y mener.
- « ou simplement quand il demande quelle version » — couvre le cas direct.

## Étape 4 — Le comportement attendu de l'agent

Sur la question test : « *Quelle version de Python utilises-tu ? Est-ce que je peux utiliser `match` / `case` ?* »

L'agent devrait :

1. Appeler `current_python_version()`.
2. Observer la sortie (par exemple "Python 3.11.4 (darwin)").
3. Répondre en mentionnant que `match/case` (PEP 634) est disponible depuis Python 3.10, et donc utilisable.

*Si l'agent n'appelle PAS le tool, c'est que la docstring n'est pas assez explicite. Reformule-la pour ajouter le mot-clé qui correspond à la question (par exemple, ajouter « `pattern matching`, `match`, `case` » parmi les cas d'usage).*

### Sortie de l'exécution

```
=====
EXERCICE 1 -- Ajout d'un tool current_python_version()
=====

[USER]:
 Quelle version de Python utilises-tu pour exécuter mon code ?
 Est-ce que je peux utiliser le pattern matching avec
 `match` / `case` ?

[AI]:
 Je vais vérifier la version de Python utilisée.
 -> tool_use: current_python_version({})

[TOOL current_python_version]:
 Python 3.12.7 (darwin)

[AI]:
 Excellente nouvelle !

 Je utilise **Python 3.12.7**, ce qui signifie que **oui, tu peux
```

tout à fait utiliser le pattern matching avec ``match`` / ``case`` !

Le pattern matching a été introduit en **Python 3.10**, donc tu as accès à cette fonctionnalité. Voici un petit exemple pour te montrer que ça fonctionne :

```
-> tool_use: run_python_code({'code':
 'def classifieur_nombre(n):\n'
 ' match n:\n'
 ' case 0: return "zéro"\n'
 ' case 1: return "un"\n'
 ' case 2: return "deux"\n'
 ' case _: return "autre nombre"\n'
 'print(classifieur_nombre(0))\n'
 'print(classifieur_nombre(1))\n'
 'print(classifieur_nombre(5))'})
```

[TOOL run\_python\_code]:

```
STDOUT:
zéro
un
autre nombre
Exit code: 0
```

[AI]:

Parfait ! Comme tu vois, le pattern matching fonctionne sans problème. Tu peux utiliser :

- Les **patterns simples** (valeurs littérales)
- Les **patterns structurels** (tuples, listes, dictionnaires)
- Les **patterns avec gardes** (conditions supplémentaires avec ``if``)
- Le **wildcard** ``_`` pour capturer n'importe quoi

## 5.2 Vérification dans LangSmith

L'arbre d'exécution doit montrer :

```
agent (LLM, tour 1)
 Output : tool_calls = [{current_python_version}]
 tools (ToolNode)
 current_python_version() → "Python 3.11.4..."
agent (LLM, tour 2)
 Output : "J'utilise Python 3.11. match/case est..."
```

Vérifie aussi l'*Input* du premier appel LLM : les **5 tools** doivent être listés dans le schéma envoyé, dont `current_python_version` avec sa docstring.

### Récapitulatif

*Ce qu'il faut retenir :*

1. Ajouter un tool est **trivial** : une fonction `@tool`, et on l'append à la liste passée à `create_agent`.
2. Pattern de composition : `tools=TOOLS + [new_tool]` évite la duplication.
3. La docstring est le **seul** levier pour orienter le modèle sur le quand-utiliser. Soigner les cas d'usage explicites.
4. Si le modèle n'appelle pas le tool attendu : c'est presque toujours un problème de docstring, pas de logique.

## 6 LangSmith pour observer un agent

### Exercice bonus 2. Tracer la boucle ReAct dans l'UI LangSmith

Configure LangSmith (variables d'environnement `LANGCHAIN_TRACING_V2`, `LANGCHAIN_API_KEY`, `LANGCHAIN_PROJECT`) et exécute l'agent sur une question **multi-tool** pour obtenir une trace riche. Ouvre la trace dans l'UI et explore l'arbre d'exécution.

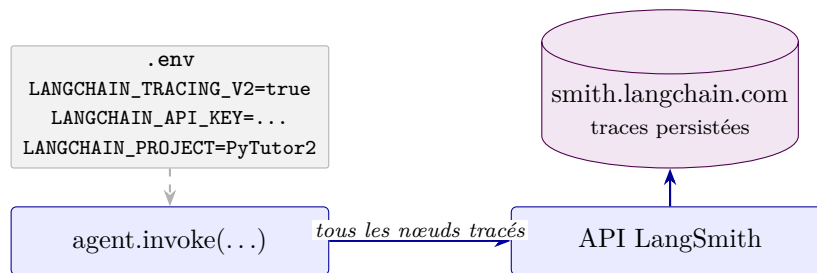
**Contrainte** : l'exercice doit vérifier programmatiquement la présence des variables d'env et afficher un mode d'emploi clair si la config manque.

#### Objectifs pédagogiques

- Configurer LangSmith pour le suivi d'un agent (variables spécifiques `LANGCHAIN_*`).
- Lire l'arbre d'exécution d'un agent dans l'UI : nœuds **agent** / **tools**, prompts exacts, tokens consommés, latence.
- Comprendre que *l'agent n'a pas de mémoire magique* : c'est l'historique qui s'accumule et est repassé au modèle à chaque tour. LangSmith le rend visible.

### 6.1 Solution

#### 6.1.1 Architecture



Quand `LANGCHAIN_TRACING_V2=true` et qu'une clé API valide est présente, *chaque* Runnable et *chaque* appel LLM sont automatiquement envoyés à LangSmith. Aucun changement de code nécessaire.

#### 6.1.2 Le code complet

```
1 import os
2 from langchain_core.messages import HumanMessage
3 from phase3_agent import build_agent
4
5
6 def exercise_2_langsmith() -> None:
7 # --- Vérification de la configuration ---
8 tracing = os.getenv("LANGCHAIN_TRACING_V2", "").lower() in {"true", "1"}
9 api_key = os.getenv("LANGCHAIN_API_KEY")
10 project = os.getenv("LANGCHAIN_PROJECT", "default")
11
12 print(f"LANGCHAIN_TRACING_V2 : {'V' if tracing else 'X'}")
13 print(f"LANGCHAIN_API_KEY : {'V' if api_key else 'X'}")
14 print(f"LANGCHAIN_PROJECT : {project}")
15
16 if not (tracing and api_key):
17 print("-> Configure ces variables dans ton .env puis relance.")
18 return
19
20 # --- Exécution d'un agent multi-tool pour une trace riche ---
```

```

21 agent = build_agent()
22 agent.invoke({
23 "messages": [HumanMessage(content=(
24 "Cherche dans la doc ce qu'est un dictionnaire en Python, "
25 "puis génère un exercice débutant sur ce sujet, "
26 "et vérifie que le code de référence est PEP 8 compliant."
27))]
28 })
29 print(f"-> Trace envoyée. Ouvre https://smith.langchain.com (projet '{project}').")

```

### 6.1.3 Explications pas à pas

#### Étape 1 — Les variables d’environnement LangSmith

| Variable             | Rôle                                                                                                                              |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| LANGCHAIN_TRACING_V2 | "true" pour activer l’envoi automatique des traces. Sans ça, les chaînes/agents s’exécutent normalement, rien n’est envoyé.       |
| LANGCHAIN_API_KEY    | Clé personnelle créée sur <a href="https://smith.langchain.com">smith.langchain.com</a> (commence par <code>lsv2_pt_...</code> ). |
| LANGCHAIN_PROJECT    | Nom du projet où grouper les traces. Ex. : "PyTutor2".                                                                            |
| LANGCHAIN_ENDPOINT   | Optionnel. <a href="https://eu.api.smith.langchain.com">https://eu.api.smith.langchain.com</a> pour un compte EU.                 |

***Note de nommage** : les variables historiques sont préfixées `LANGCHAIN_` ; la nouvelle version unifiée est préfixée `LANGSMITH_`. Les deux sont reconnues. Cohérence avec le reste du projet recommandée.*

#### Étape 2 — Une question multi-tool pour une trace riche

Pour avoir une trace pédagogiquement intéressante, choisis une question qui force l’agent à enchaîner plusieurs tools :

« Cherche dans la doc ce qu’est un dictionnaire en Python, puis génère un exercice débutant sur ce sujet, et vérifie que le code de référence est PEP 8 compliant. »

Cette question demande explicitement :

1. `search_python_docs` (« cherche dans la doc »).
2. `generate_exercise` (« génère un exercice »).
3. `check_pep8` (« vérifie PEP 8 »).

L’agent va donc faire **3 itérations** de la boucle ReAct, ce qui donne une trace très lisible.

#### Étape 3 — Ce qu’on voit dans l’UI LangSmith

Dans [smith.langchain.com](https://smith.langchain.com), dans ton projet :

1. Une trace nommée **LangGraph** apparaît : l’agent *est* un graphe LangGraph.
2. L’**arbre d’exécution** montre l’alternance **agent** (LLM) / **tools** (ToolNode) sur 3 cycles.
3. Chaque **agent** (model call) expose :
  - *Input* : système + historique + schémas des tools.
  - *Output* : la décision (`tool_calls`) ou la réponse finale.
4. Chaque **tools** expose les noms des tools appelés, leurs arguments, et leurs sorties intégrales.
5. Tokens et latence *par appel* sont visibles, agrégés sur la trace racine.

## Étape 4 — L’insight clé : l’historique grossit

À chaque tour du modèle, regarde l’onglet Input dans la trace. Tu vois **tout l’historique précédent** qui est repassé au LLM. L’agent n’a pas de mémoire magique : c’est juste l’historique qui grossit, et que LangChain repasse à chaque appel. C’est aussi pourquoi le coût en tokens d’entrée augmente à chaque tour.

## 6.2 Vérification dans LangSmith

(Cet exercice EST la vérification.)

Quatre choses à pointer dans la trace pour bien comprendre :

- L’**alternance** agent → tools → agent → ...
- Le même **system\_prompt** en tête de chaque appel agent (constant).
- L’**historique** qui grandit dans l’input de chaque appel agent.
- La **dernière** étape : un agent qui produit un Output **sans** tool\_calls — c’est la réponse finale.

### Récapitulatif

Ce qu’il faut retenir :

1. 3 variables d’env : `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_API_KEY=...`, `LANGCHAIN_PROJECT=...`
2. Tracing automatique : aucune modification de code une fois activé.
3. Une trace d’agent montre l’**alternance** agent/tools/agent... jusqu’à la décision finale.
4. L’historique grossit visible dans l’Input de chaque appel LLM successif. C’est l’observation pédagogique fondamentale.
5. LangSmith expose tokens et latence par appel, agrégés sur la trace racine : indispensable pour optimiser un agent.

## 7 Tool avec paramètre timeout configurable

### Exercice bonus 3. Exposer un paramètre par défaut au modèle

Modifie `run_python_code` pour accepter un paramètre `timeout_seconds: int = 5`. La docstring doit guider le modèle sur *quand* surcharger la valeur par défaut (calcul long manifeste) et *quand pas* (la majorité des cas). Teste avec deux scénarios : un code rapide et un code volontairement long.

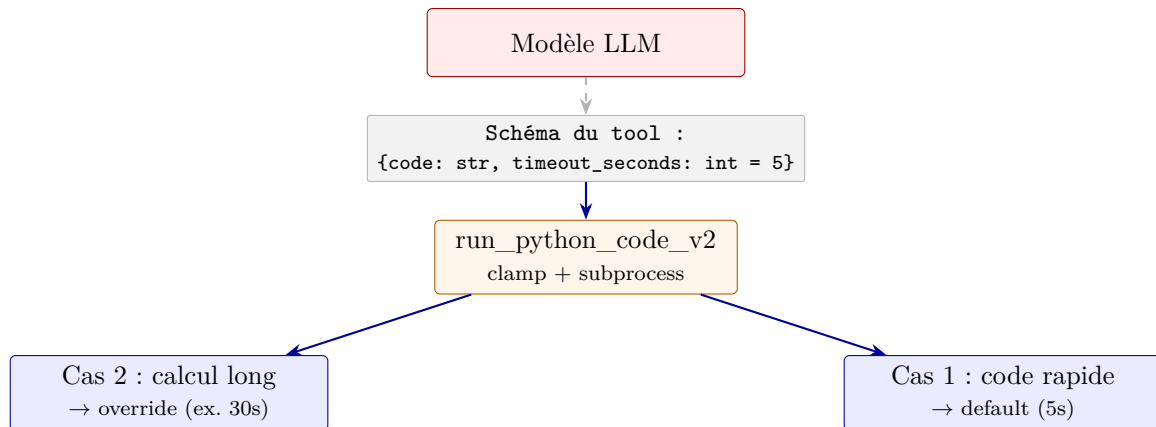
**Contrainte** : le timeout doit être *clampé* côté serveur ( $1 \leq t \leq 60$  secondes) pour empêcher des valeurs absurdes.

#### Objectifs pédagogiques

- Comprendre que les paramètres avec valeur par défaut sont **exposés au modèle** dans le schéma JSON du tool.
- Voir comment la docstring conditionne la décision du modèle d’override ou non.
- Mettre en place une **défense en profondeur** : une consigne dans la docstring *plus* un clamping côté code.

## 7.1 Solution

### 7.1.1 Architecture



Le modèle voit la signature complète, lit la docstring, et décide à chaque appel s'il garde la valeur par défaut ou la surcharge.

### 7.1.2 Le code complet

```
1 import subprocess
2 import sys
3 import tempfile
4 from pathlib import Path
5
6 from langchain_core.tools import tool
7
8
9 @tool
10 def run_python_code_v2(code: str, timeout_seconds: int = 5) -> str:
11 """Exécute du code Python et renvoie stdout + stderr.
12
13 Args:
14 code: Le code Python à exécuter, complet et autonome.
15 timeout_seconds: Délai max avant interruption. Défaut : 5 secondes.
16 Augmente cette valeur SEULEMENT si le code requiert manifestement
17 un calcul long (entraînement, traitement de gros datasets, etc.).
18 Maximum raisonnable : 60 secondes.
19 """
20 timeout_seconds = max(1, min(timeout_seconds, 60)) # clamp défensif
21
22 with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f:
23 f.write(code)
24 path = f.name
25 try:
26 result = subprocess.run(
27 [sys.executable, path],
28 capture_output=True, text=True, timeout=timeout_seconds,
29)
30 parts = []
31 if result.stdout:
32 parts.append(f"STDOUT:\n{result.stdout}")
33 if result.stderr:
34 parts.append(f"STDERR:\n{result.stderr}")
35 parts.append(f"Exit code: {result.returncode} (timeout utilisé:
36 ↪ {timeout_seconds}s)")
```

---

```

36 return "\n\n".join(parts)
37 except subprocess.TimeoutExpired:
38 return (
39 f"ERREUR: timeout après {timeout_seconds} secondes. "
40 "Si tu pensais que le code devait être long, augmente timeout_seconds. "
41 "Sinon, il y a sans doute une boucle infinie ou une récursion non terminale."
42)
43 finally:
44 Path(path).unlink(missing_ok=True)

```

---

### 7.1.3 Explications pas à pas

#### Étape 1 — Le paramètre avec valeur par défaut

---

```

1 def run_python_code_v2(code: str, timeout_seconds: int = 5) -> str:

```

---

Le décorateur `@tool` extrait *toute* la signature et la traduit en JSON Schema :

---

```

1 {
2 "type": "object",
3 "properties": {
4 "code": {"type": "string"},
5 "timeout_seconds": {"type": "integer", "default": 5},
6 },
7 "required": ["code"], # timeout_seconds N'EST PAS requis
8 }

```

---

Ce JSON Schema est envoyé au LLM. Le modèle peut donc **choisir** de remplir `timeout_seconds`, ou de laisser le tool utiliser le défaut.

#### Étape 2 — La docstring guide la décision

C'est la docstring du paramètre qui pilote le comportement :

*« Augmente cette valeur **SEULEMENT** si le code requiert manifestement un calcul long (entraînement, traitement de gros datasets, etc.). Maximum raisonnable : 60 secondes. »*

Trois leviers :

- « **SEULEMENT si** » en capitales : signal fort que c'est une exception, pas la règle.
- **Exemples concrets** : « entraînement, gros datasets ». Donne au LLM des templates de cas qui justifient l'override.
- **Maximum nommé** : « 60 secondes » : le LLM ne devrait pas dépasser cette borne, mais on a aussi un clamp côté serveur au cas où.

#### Étape 3 — Le clamp défensif côté Python

---

```

1 timeout_seconds = max(1, min(timeout_seconds, 60))

```

---

Pourquoi double protection (prompt + clamp) ?

- Le modèle peut **halluciner** une valeur (1000 secondes, ou même négative).
- Le modèle peut être **exploité** : une question pourrait chercher à passer `timeout_seconds=99999` pour bloquer le serveur.



- En production, on ne fait jamais confiance au LLM pour des valeurs critiques : **défense en profondeur**.

*La règle générale : tout paramètre numérique exposé à un LLM doit avoir des bornes explicites côté serveur. La docstring guide ; le clamp protège.*

## Étape 4 — Les deux cas attendus

### Cas 1 — code rapide :

```
1 "Exécute ce code : print(sum(range(100)))"
```

L'agent appelle `run_python_code_v2(code="...")` sans spécifier `timeout_seconds` — il utilise le défaut de 5s. Dans l'historique, le `tool_call` contient un seul argument : `{"code": "..."}`.

### Cas 2 — calcul long (somme des premiers jusqu'à 5M) :

L'utilisateur précise « ça met une bonne dizaine de secondes ». L'agent *lit* cette information, comprend qu'il dépasse le défaut, et appelle :

```
1 run_python_code_v2(code="...", timeout_seconds=30)
```

*C'est une preuve que la docstring conditionne le comportement réel du modèle. La phrase « Augmente cette valeur SEULEMENT si... » est lue et appliquée. Tu peux tester en retirant cette phrase et observer que le modèle, soit override systématiquement (overshoot), soit ne l'override jamais (timeout au cas 2).*

#### Sortie de l'exécution

```
=====
EXERCICE 3 -- Tool avec paramètre timeout_seconds configurable
=====

--- Cas 1 : code rapide (timeout par défaut attendu) ---

[USER]:
Exécute ce code pour me dire la sortie :
```python
print(sum(range(100)))
```

[AI]:
-> tool_use: run_python_code_v2({'code':
 'print(sum(range(100)))'})
 (pas de timeout_seconds -> défaut 5s)

[TOOL run_python_code_v2]:
 STDOUT:
 4950
 Exit code: 0 (timeout utilisé: 5s)

[AI]:
 La sortie est **4950**.

 Ce code calcule la somme de tous les nombres de 0 à 99 (car
 `range(100)` génère les entiers de 0 à 99 inclus).
 Mathématiquement, c'est : 0 + 1 + 2 + ... + 99 = **4950**.
```

```

--- Cas 2 : calcul long, l'agent doit augmenter le timeout ---

[USER]:
J'ai écrit un code qui calcule la somme des nombres premiers
jusqu'à 5 millions. Ça met une bonne dizaine de secondes à tourner
sur ma machine. Tu peux l'exécuter pour vérifier le résultat ?

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

total = sum(n for n in range(2, 5_000_000) if is_prime(n))
print(total)
```

[AI]:
Je vais exécuter ton code. Comme tu l'as mentionné, ça va prendre
un peu de temps, donc je vais augmenter le délai d'exécution.
-> tool_use: run_python_code_v2({
 'code': '...',
 'timeout_seconds': 30 <-- augmenté explicitement
})

[TOOL run_python_code_v2]:
STDOUT:
838596693108
Exit code: 0 (timeout utilisé: 30s)

[AI]:
Parfait ! Ton code fonctionne correctement et donne le résultat :
838 596 693 108

Ton algorithme est correct, mais je peux te montrer comment
l'optimiser significativement avec le Crible d'Eratosthène,
qui trouve tous les nombres premiers jusqu'à une limite en une
seule passe (probablement 10-20x plus rapide).

OBSERVATIONS :
- Au cas 1, l'agent appelle le tool SANS spécifier timeout_seconds
 (il garde la valeur par défaut 5s).
- Au cas 2, l'agent voit dans ta question que le code prend ~10s,
 et passe explicitement timeout_seconds=30 (ou similaire).
- C'est UNE preuve que la docstring conditionne le comportement :
 la phrase 'Augmente cette valeur SEULEMENT si...' est lue et
 appliquée par le modèle.

```

## 7.2 Vérification dans LangSmith

Dans la trace, examine l'onglet *Tool calls* du *AIMessage* qui invoque `run_python_code_v2` :

- **Cas 1** : `args = {"code": "..."}`. La clé `timeout_seconds` **n'apparaît pas** → défaut utilisé.
- **Cas 2** : `args = {"code": "...", "timeout_seconds": 30}`. L'override est explicite.

Le *ToolMessage* en sortie contient la trace exacte du timeout utilisé ("`Exit code: 0 (timeout utilisé: 30s)`"), ce qui permet de vérifier que le clamp côté serveur a bien laissé passer la valeur.

## Récapitulatif

*Ce qu'il faut retenir :*

1. Les **paramètres avec défaut** sont exposés au LLM dans le JSON Schema. Le LLM choisit de les remplir ou pas.
2. La **description** du paramètre dans la docstring est ce qui guide le modèle pour quand override.
3. **Défense en profondeur** : prompt qui guide + clamp côté serveur. Ne jamais faire confiance aux valeurs numériques produites par le LLM.
4. Toujours répercuter la valeur effectivement utilisée dans la sortie du tool (« timeout utilisé : 30s ») : ça aide l'agent à interpréter le résultat.

## 8 Tool qui appelle un sous-LLM (evaluate\_student\_answer)

### Exercice bonus 4. Composer un agent et un sous-LLM dans un tool

Crée un tool `evaluate_student_answer(question, answer)` qui prend une question et la réponse d'un élève, et renvoie une évaluation structurée (correct/incorrect, note, feedback, concepts manquants). Le tool doit utiliser **en interne** une mini-chaîne LCEL avec `with_structured_output(AnswerEvaluation)`.

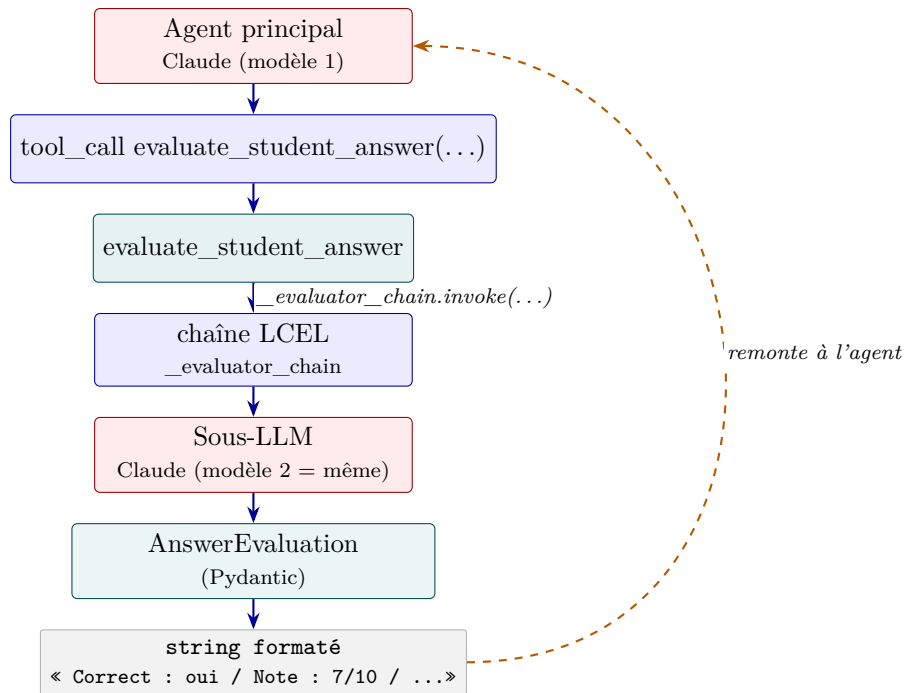
**Contrainte** : la mini-chaîne LCEL est construite au niveau du module (une fois pour toute) pour éviter de la recréer à chaque appel du tool.

#### Objectifs pédagogiques

- Maîtriser le pattern « tool qui appelle un autre LLM en interne » : c'est la composition d'agents au sens large.
- Utiliser `with_structured_output` avec un schéma Pydantic riche pour produire des évaluations *exploitables*.
- Comprendre l'architecture en **couches** : agent principal → tool → sous-chaîne LCEL → sous-LLM.
- Observer le coût : 2 appels LLM dans un seul tour d'agent.

## 8.1 Solution

### 8.1.1 Architecture



Vu de l'agent principal, c'est un tool comme un autre. Vu de l'intérieur, c'est un **deuxième appel LLM** caché.

### 8.1.2 Le code complet

```
1 from pydantic import BaseModel, Field
2 from langchain_core.tools import tool
3 from langchain_core.prompts import ChatPromptTemplate
4 from config import get_model
5
6
7 class AnswerEvaluation(BaseModel):
8 """Évaluation pédagogique d'une réponse d'élève."""
9
10 is_correct: bool = Field(
11 description="True si la réponse est essentiellement correcte, "
12 "False si elle contient une erreur substantielle."
13)
14 score: int = Field(
15 description="Note de 0 à 10. 0 = totalement faux, 10 = parfait. "
16 "Tiens compte de la précision technique ET de la clarté."
17)
18 feedback: str = Field(
19 description="Feedback pédagogique en 2-3 phrases pour l'élève. "
20 "Souligne ce qui est juste, puis ce qu'il faut corriger ou préciser."
21)
22 missing_concepts: list[str] = Field(
23 description="Concepts importants que la réponse aurait dû mentionner mais omet. "
24 "Liste vide si la réponse est complète."
25)
26
27
28 # La mini-chaîne LCEL utilisée à l'intérieur du tool. On la construit
```

```

29 # une seule fois en module-level pour éviter de la recréer à chaque appel.
30 _eval_prompt = ChatPromptTemplate.from_messages([
31 ("system",
32 "Tu es un évaluateur pédagogique de réponses Python. Sois exigeant "
33 "techniquement mais bienveillant dans la formulation."),
34 ("human",
35 "Question posée à l'élève :\n{question}\n\n"
36 "Réponse de l'élève :\n{answer}\n\n"
37 "Évalue cette réponse."),
38])
39
40 _evaluator_chain = (
41 _eval_prompt
42 | get_model().with_structured_output(AnswerEvaluation)
43)
44
45
46 @tool
47 def evaluate_student_answer(question: str, answer: str) -> str:
48 """Évalue la réponse d'un élève à une question technique sur Python.
49
50 À utiliser quand un élève a écrit une réponse et que tu veux la noter
51 objectivement plutôt que de te contenter d'un "oui" ou "non" qualitatif.
52 Renvoie un score, un feedback, et la liste des concepts manquants.
53
54 Args:
55 question: La question posée à l'élève.
56 answer: La réponse complète de l'élève à évaluer.
57 """
58 # Sous-LLM : on appelle une mini-chaîne LCEL avec sortie structurée.
59 result = _evaluator_chain.invoke({"question": question, "answer": answer})
60
61 # On reformate l'objet Pydantic en string pour l'agent.
62 # (Les tools renvoient toujours du texte, c'est leur protocole.)
63 missing = ", ".join(result.missing_concepts) if result.missing_concepts else
64 ↪ "(aucun)"
65 return (
66 f"Correct : {'oui' if result.is_correct else 'non'}\n"
67 f>Note : {result.score}/10\n"
68 f"Concepts manquants : {missing}\n"
69 f"Feedback : {result.feedback}"
70)

```

### 8.1.3 Explications pas à pas

#### Étape 1 — Le schéma Pydantic d'évaluation

`AnswerEvaluation` contient 4 champs orientés *pédagogie* : un booléen pour le verdict net, un score chiffré, un feedback en langage naturel, et une liste de concepts manquants.

Le choix des types n'est pas neutre :

- `is_correct`: `bool` : verdict binaire utile en aval (par exemple pour filtrer les bonnes réponses).
- `score`: `int` : granularité 0-10, exploitable pour agrégation et stats.
- `feedback`: `str` : texte à montrer à l'élève.
- `missing_concepts`: `list[str]` : liste itérable, pas une chaîne JSON. Permet de calculer du *coverage* sur plusieurs réponses.

## Étape 2 — La mini-chaîne LCEL construite *une fois*

```
1 _eval_prompt = ChatPromptTemplate.from_messages([...])
2 _evaluator_chain = _eval_prompt | get_model().with_structured_output(AnswerEvaluation)
```

**Pourquoi au niveau du module ?** `get_model()` crée une instance du client (charge la config, ouvre éventuellement des connexions). On la construit une fois à l'import et on la réutilise. Sinon, chaque appel du tool recréerait l'instance — coûteux et inutile.

*Pattern important : tout ce qui est coûteux à construire et réutilisable (modèle, chaîne, retriever, embeddings...) vit au niveau du module. Les fonctions et tools ne font que .invoke() sur ces objets.*

## Étape 3 — Le tool reformate en string

Le tool reçoit deux strings (question, answer), invoque la chaîne, et **reformate** l'objet Pydantic en string :

```
1 result = _evaluator_chain.invoke({"question": question, "answer": answer})
2 # result est un objet AnswerEvaluation
3
4 return (
5 f"Correct : {'oui' if result.is_correct else 'non'}\n"
6 f>Note : {result.score}/10\n"
7 ...
8)
```

**Pourquoi reformater ?** Le protocole des tools est : *ils renvoient une string*. Cette string devient le contenu d'un ToolMessage dans l'historique, ensuite *relu* par l'agent principal pour formuler sa réponse à l'utilisateur.

## Étape 4 — L'architecture en couches

Quand l'agent principal traite la question « *Évalue cette réponse d'élève* », l'arbre d'exécution a quatre niveaux :

1. **Agent principal (Claude)** : décide d'appeler `evaluate_student_answer(question, answer)`.
2. **Tool `evaluate_student_answer`** : Python exécute la fonction.
3. **Sous-LLM (Claude à nouveau)** : appelé en interne par la mini-chaîne. Reçoit son *propre* prompt système et renvoie un `AnswerEvaluation` validé par Pydantic.
4. **Le tool reformate** l'objet en string et la rend à l'agent principal.
5. **L'agent principal** intègre le résultat et formule sa réponse à l'utilisateur.

C'est **2 appels LLM** dans un seul tour d'agent : un par l'agent principal pour décider, un par le sous-LLM pour évaluer.

## Étape 5 — Pourquoi ce pattern est important

Le pattern « tool qui appelle un sous-LLM » résout des problèmes fréquents :

- **Compétence spécialisée** : l'agent principal n'est pas un évaluateur ; il *délègue* à un LLM dédié avec son prompt d'évaluation.
- **Sortie structurée garantie** : l'agent principal manipule `AnswerEvaluation` via le tool, donc `score` est toujours un `int` entre 0 et 10, jamais une string ambiguë.

- **Réutilisabilité** : la mini-chaîne peut être appelée directement ailleurs (interface enseignant) sans passer par l'agent.

#### Sortie de l'exécution

```
=====
EXERCICE 4 -- Tool 'evaluate_student_answer' (sous-LLM caché)
=====

[USER]:
Un élève a répondu à une question d'examen. Évalue-la pour moi et
formule un commentaire que je pourrais lui envoyer.

Question : A quoi sert le mot-clé `yield` en Python et en quoi
diffère-t-il de `return` ?

Réponse de l'élève : << `yield` permet de retourner plusieurs
valeurs depuis une fonction, contrairement à `return` qui n'en
retourne qu'une seule. >>

[AI]:
Je vais évaluer cette réponse de manière objective.
-> tool_use: evaluate_student_answer({
 'question': 'A quoi sert le mot-clé `yield`...',
 'answer': '`yield` permet de retourner...'
})

[TOOL evaluate_student_answer]:
Correct : non
Note : 4/10
Concepts manquants :
- Générateurs et itérateurs
- Lazy evaluation (évaluation paresseuse)
- Sauvegarde de l'état d'exécution avec yield
- Efficacité mémoire des générateurs
- Différence entre retourner une liste et utiliser yield
Feedback : Tu as identifié une différence importa...

[AI]:
Commentaire à envoyer à l'élève :

Evaluation : 4/10

Bonne intuition, mais ta réponse est incomplète et imprécise.
Voici ce qui manque :

Ce que tu as bien compris
Tu as raison que `yield` permet de retourner plusieurs valeurs, ce
qui est une différence avec `return`.

Ce qui manque
La vraie différence entre `yield` et `return` ne porte pas
seulement sur le nombre de valeurs, mais sur comment et
quand elles sont retournées :

1. `yield` crée un générateur : une fonction qui retourne les
valeurs une à une, à la demande (lazy evaluation), sans
tout calculer d'un coup.
2. `return` retourne tout immédiatement : la fonction s'arrête
et retourne une valeur unique (ou une liste si tu veux
plusieurs valeurs).
3. Avantage mémoire : avec `yield`, tu économises de la
mémoire car les valeurs ne sont générées que quand on les
demande.
```

```

Exemple pour clarifier
```python
# Avec return : tout est calculé et stocké en mémoire
def nombres_return():
    return [1, 2, 3, 4, 5]

# Avec yield : les valeurs sont générées à la demande
def nombres_yield():
    yield 1; yield 2; yield 3; yield 4; yield 5
...

**A revoir** : les concepts de générateurs, itérateurs et
lazy evaluation pour bien maîtriser `yield`.

```

----- ARCHITECTURE EN COUCHES A VISUALISER :

1. AGENT PRINCIPAL (Claude) : décide d'appeler `evaluate_student_answer`.
2. TOOL `evaluate_student_answer` : Python exécute la fonction.
3. SOUS-LLM (Claude à nouveau) : appelé en interne par la mini-chaîne.
Reçoit son propre prompt, renvoie un objet Pydantic structuré.
4. Le tool reformate l'objet en string et le rend à l'agent.
5. L'agent intègre le résultat et formule sa réponse à l'utilisateur.

-> 2 appels LLM dans ce tour, pour 1 invocation d'agent.
-> Si tu actives LangSmith, tu verras les 2 traces emboîtées.

8.2 Vérification dans LangSmith

L'arbre d'exécution montre les **traces emboîtées** :

```

agent (LLM #1) ← Claude décide
Output: tool_calls = [evaluate_student_answer]
tools (ToolNode)
evaluate_student_answer(question, answer)
RunnableSequence (_evaluator_chain)
ChatPromptTemplate
ChatAnthropic (LLM #2) → AnswerEvaluation
agent (LLM #3) ← Claude répond

```

Tu peux cliquer sur chaque `ChatAnthropic` pour voir les prompts *différents* : l'un est le `system_prompt` de l'agent principal, l'autre est celui de l'évaluateur.

Récapitulatif

Ce qu'il faut retenir :

1. Pattern **tool** → **sous-LLM** : un tool peut contenir une chaîne LCEL complète, avec son propre prompt et sortie structurée.
2. Construire les objets coûteux (modèle, chaîne) au niveau du **module**, pas dans la fonction du tool.
3. Les tools renvoient toujours du **texte** : reformater l'objet Pydantic en string compréhensible pour l'agent.
4. Coût : **2 appels LLM** dans un seul tour d'agent. Visible en clair dans LangSmith (traces emboîtées).
5. Cas d'usage : déléguer une compétence spécialisée (évaluation, classification, extraction) tout en gardant un agent généraliste en orchestrateur.

9 Streaming avec `agent.stream()`

Exercice bonus 5. Streamer un agent en deux modes complémentaires

Passer de `agent.invoke()` à `agent.stream()` pour obtenir une UX réactive. Expérimente avec deux modes : `stream_mode="updates"` (chaque étape du graphe émet un chunk au moment où le nœud termine) et `stream_mode="messages"` (chaque token du modèle est émis individuellement, effet machine à écrire).

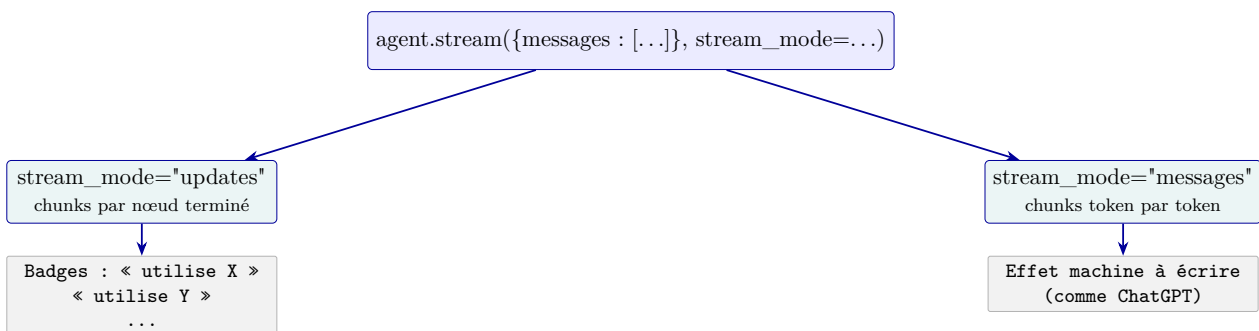
Contrainte : dans le mode `messages`, filtrer pour ne streamer que les tokens du nœud `agent` (pas ceux d'éventuels sous-LLM dans des `tools`).

Objectifs pédagogiques

- Connaître les trois modes de `agent.stream()` : `updates`, `messages`, `values`.
- Savoir lire un chunk `updates` : c'est un dict `{node_name: state_diff}`.
- Savoir filtrer le streaming `messages` par nœud (via la `metadata`).
- Combiner les deux modes pour une UX moderne (badges d'étapes + machine à écrire).

9.1 Solution

9.1.1 Architecture



Les deux modes **ne sont pas exclusifs** : en production, on les combine. `updates` pour afficher des badges au-dessus du chat (« l'agent utilise `search_python_docs...` ») ; `messages` pour le texte qui s'écrit progressivement.

9.1.2 Le code complet

```
1 from langchain_core.messages import HumanMessage
2 from phase3_agent import build_agent
3
4
5 def exercise_5_updates() -> None:
6     """Streaming au niveau des ÉTAPES (mode 'updates')."""
7     agent = build_agent()
8
9     for chunk in agent.stream(
10         {"messages": [HumanMessage(content=(
11             "Cherche dans la doc ce qu'est un itérable, "
12             "puis génère un exercice débutant sur ce sujet."
13         ))]},
14         stream_mode="updates",
15     ):
16         # chunk est un dict {nom_du_nœud: {clés_modifiées: valeurs}}.
```

```

17     # Avec create_agent, on a deux noeuds principaux : 'agent' et 'tools'.
18     for node_name, node_update in chunk.items():
19         messages = node_update.get("messages", [])
20         for msg in messages:
21             if hasattr(msg, "tool_calls") and msg.tool_calls:
22                 for tc in msg.tool_calls:
23                     print(f" [{node_name}] -> appel
24                             ↪ {tc['name']}({list(tc['args'].keys())})")
25             elif hasattr(msg, "name") and msg.name:
26                 # ToolMessage : a un .name pointant vers le tool qui a renvoyé
27                 preview = (msg.content or "")[:80].replace("\n", " ")
28                 print(f" [{node_name}] <- retour de {msg.name}: {preview}...")
29             elif msg.content:
30                 preview = msg.content[:120].replace("\n", " ")
31                 print(f" [{node_name}] >> {preview}...")
32
33 def exercise_5_tokens() -> None:
34     """Streaming token par token (mode 'messages')."""
35     agent = build_agent()
36
37     # Mode 'messages' : on reçoit des tuples (chunk_message, metadata).
38     for chunk, metadata in agent.stream(
39         {"messages": [HumanMessage(content=(
40             "Explique-moi en deux phrases ce qu'est une compréhension de liste."
41         ))]},
42         stream_mode="messages",
43     ):
44         # On ne stream que le texte produit par le noeud 'agent' (pas les tools)
45         if metadata.get("langgraph_node") == "agent" and chunk.content:
46             text = chunk.content if isinstance(chunk.content, str) else "".join(
47                 b.get("text", "") for b in chunk.content if isinstance(b, dict)
48             )
49             print(text, end="", flush=True)
50     print()

```

9.1.3 Explications pas à pas

Étape 1 — Les trois modes de streaming

Mode	Ce qu'on reçoit
updates	Un chunk par mise à jour de l'état du graphe (= un nœud termine). Format : {node_name: state_diff}. Idéal pour afficher « l'agent a appelé tel tool » en temps réel.
messages	Un chunk par token produit par un modèle. Format : tuple (chunk_msg, metadata). Idéal pour l'effet machine à écrire.
values	L'état complet du graphe à chaque étape (très verbeux, utile en debug).

Étape 2 — Mode updates : chunks par étape

```

1 for chunk in agent.stream({"messages": [...]}, stream_mode="updates"):
2     for node_name, node_update in chunk.items():
3         # node_name est typiquement 'agent' ou 'tools'
4         messages = node_update.get("messages", [])

```

```
5 # messages contient ce que ce nœud A AJOUTÉ à l'état
```

Avec `create_agent`, deux nœuds principaux émettent des updates :

- `agent` : produit des `AIMessage` (avec ou sans `tool_calls`).
- `tools` : produit des `ToolMessage` (résultats des tools).

Dans la boucle, on inspecte le contenu pour afficher : appel de tool, retour de tool, ou réponse textuelle.

Étape 3 — Mode messages : tokens du LLM

```
1 for chunk, metadata in agent.stream({"messages": [...]} , stream_mode="messages"):
2     if metadata.get("langgraph_node") == "agent" and chunk.content:
3         print(chunk.content, end="", flush=True)
```

Deux subtilités :

- Le chunk est un `AIMessageChunk` (ou un autre type de message). `chunk.content` peut être une `str` ou une liste de blocs (selon la version de l'API). Le code gère les deux cas.
- Le `metadata.langgraph_node` indique de *quel* nœud du graphe provient le token. On filtre sur `"agent"` pour ne pas streamer accidentellement les tokens d'un éventuel sous-LLM caché dans un tool.

Étape 4 — `print(..., end="", flush=True)` : le secret

Pour que les tokens s'écrivent en temps réel sur le terminal :

- `end=""` : pas de saut de ligne après chaque chunk.
 - `flush=True` : force l'écriture immédiate (sans ça, Python bufferise par ligne ou par bloc de 4ko).
- C'est le même pattern que pour le streaming *chains* de la Phase 1.

Étape 5 — Combiner les deux modes en production

Dans une vraie UI :

- En **haut** du chat : badges « l'agent réfléchit... », « utilise `search_python_docs...` » — issus du mode `updates`.
- Dans la **bulle de réponse** : texte qui s'écrit progressivement — issu du mode `messages`.

LangChain permet d'obtenir les deux en parallèle avec `stream_mode=["updates", "messages"]` :

```
1 for kind, chunk in agent.stream(
2     {"messages": [...]} ,
3     stream_mode=["updates", "messages"], # mode multiple
4 ):
5     if kind == "updates":
6         # afficher badge
7         pass
8     elif kind == "messages":
9         # afficher token
10        pass
```

***Règle pratique** : en production, toujours streamer. Une UX qui attend 8 secondes en boîte noire avant d'afficher la réponse complète est perçue comme cassée — même si la durée totale est identique au streaming.*

Sortie de l'exécution

```
=====
EXERCICE 5a -- Streaming par étapes (stream_mode='updates')
=====

Question envoyée. Affichage des étapes au fil de l'eau :

[model] -> appel search_python_docs(['query'])
[model] -> appel generate_exercise(['topic', 'difficulty'])
[OK] Rechargement de l'index depuis ./chroma_db
[tools] <- retour de search_python_docs: Source:
        https://docs.python.org/3/glossary.html or the script
        being run has fini...
[tools] <- retour de generate_exercise: Exercice sur 'itérables'
        (niveau débutant) : Code de référence : # Exemple 1 :...
[model] [chat] ## Définition d'un itérable
        D'après la documentation officielle Python, un
        itérable est :
        > Un objet capable ...

=====
EXERCICE 5b -- Streaming token par token (stream_mode='messages')
=====

Réponse qui s'affiche au fur et à mesure (effet machine à écrire) :

[... tokens consommés un par un, sortie typewriter ...]

-----
QUAND UTILISER QUEL MODE :
- 'updates' : UI qui montre 'l'agent réfléchit / l'agent utilise X'.
              Idéal pour les workflows long-running où
              l'utilisateur attendrait sinon dans le vide.
- 'messages' : effet machine à écrire, comme ChatGPT et Claude.ai.
              Indispensable pour une UX de chat moderne.
- On combine souvent les deux : on stream les tokens du texte
  final, et on affiche des badges 'utilise tool X' pour les
  étapes intermédiaires.
```

9.2 Vérification dans LangSmith

`agent.stream()` produit la **même trace** que `agent.invoke()` dans LangSmith : les nœuds sont tracés au moment où ils terminent. La seule différence est côté client : on consomme incrémentalement au lieu d'attendre la fin.

LangSmith expose la métrique Time to first token (TTFT) dans les runs streamés. Sur un agent à 2 itérations ReAct, le TTFT est de l'ordre du `search_python_docs` (rapide) ou du premier `ChatAnthropic` (variable). Une UI streamée affiche le premier contenu en ~ 300 ms au lieu de ~ 5 s.

Récapitulatif

Ce qu'il faut retenir :

1. `agent.stream(input, stream_mode=...)` a 3 modes : `updates` (par nœud), `messages` (par token), `values` (état complet).
2. Mode `updates` : `chunks {node_name: state_diff}`, idéal pour les badges « l'agent fait X ».
3. Mode `messages` : `tuples (chunk_msg, metadata)`. Filtrer sur `metadata["langgraph_node"] == "agent"` pour exclure les sous-LLM.
4. `print(..., end="", flush=True)` reste la condition sine qua non du streaming visible côté terminal.

5. En production, on **combine** les deux modes (`stream_mode=["updates", "messages"]`) pour une UX moderne avec badges d'étapes et machine à écrire.